

JAVA

OOP Part 2: Polimorfismo, Composição & Encapsulamento

Polimorfismo • Composição • Encapsulamento • instanceof • Packages

Apostila completa com os 3 pilares da OOP,
exemplos de código e mini-project Bill's Burger.

CONTEÚDO DA APOSTILA

- 01 Introdução — Os 3 Pilares da OOP
- 02 Composição (HAS-A) — Construindo com partes
- 03 Por que preferir Composição?
- 04 Encapsulamento — Proteção de dados
- 05 Princípios e Boas Práticas
- 06 Polimorfismo — Muitas formas em runtime
- 07 Código genérico e Factory Method
- 08 instanceof & Casting + Pattern Matching
- 09 var e LVTI (Java 10+)
- 10 Packages & Import
- 11 Mini-Projects: Smart Kitchen, Printer, Car
- 12 Mini-Project Master: Bill's Burger

1

Introdução — Os 3 Pilares da OOP

Composição, Encapsulamento e Polimorfismo em conjunto

A OOP Part 1 cobriu os fundamentos: *classes, objetos, construtores, herança e as bases do polimorfismo*. Esta apostila aprofunda os **3 pilares restantes**: *Composição, Encapsulamento e Polimorfismo* — mostrando como eles trabalham juntos para produzir código robusto, extensível e fácil de manter.

Os 3 Pilares — Uma Visão Integrada

Composição define como objetos são construídos a partir de outros objetos (relação HAS-A: *tem um*). Um `MealOrder` tem um `Burger`, tem uma bebida, tem um acompanhamento. Em vez de herdar tudo de uma única superclasse, você monta o objeto a partir de peças especializadas — mais flexível e menos acoplado.

Encapsulamento protege o estado interno do objeto: os campos são `private` e acessados apenas através de métodos controlados. O mundo externo interage com o objeto apenas pela sua *interface pública* — sem conhecer os detalhes internos. Mudanças internas não quebram o código cliente.

Polimorfismo permite que o mesmo código funcione de maneira diferente para tipos distintos em runtime. Um método `getAdjustedPrice()` chamado em uma variável do tipo `Item` pode executar a implementação de `Burger` ou de `DeluxeBurger` — dependendo do tipo real do objeto.

```
public class MealOrder {           // ← Composição (HAS-A)
    private Burger burger;         // MealOrder TEM um Burger
    private Item drink;            // MealOrder TEM um Item (drink)
    private Item sideItem;         // MealOrder TEM um Item (side)
                                   // ← Encapsulamento: campos private
    public void addToppingToBurger(String topping) {
        burger.addTopping(topping); // delega – não expõe burger
    }
    public double getTotalPrice() {
        // ← Polimorfismo: getAdjustedPrice() comporta-se diferente
        // para Burger, DeluxeBurger, drink e sideItem em runtime
        return burger.getAdjustedPrice()
            + drink.getAdjustedPrice()
            + sideItem.getAdjustedPrice();
    }
}
```

■ Pilares em conjunto

Os três pilares trabalham juntos — raramente você usará apenas um em isolamento. Um bom design de classe tipicamente aplica os três simultaneamente.

2**Composição**

HAS-A vs IS-A, design flexível e por que preferir composição

Herança IS-A vs Composição HAS-A

Herança define uma relação **IS-A**: *Dog IS-A Animal*. A subclasse é *um tipo* da superclasse — herda toda a sua identidade. **Composição**, por sua vez, define uma relação **HAS-A**: *PersonalComputer HAS-A Motherboard*. O objeto *contém* outros objetos como partes — sem herdar nenhuma de suas identidades.

A regra geral da engenharia de software é: **prefira composição antes de herança**. Isso não significa nunca usar herança — significa avaliar cuidadosamente se a relação IS-A faz sentido semanticamente antes de usá-la. Você pode — e frequentemente deve — usar os dois no mesmo design: herança para identidade, composição para funcionalidade.

```
// IS-A via herança:
public class Product { String manufacturer, model; }
public class PersonalComputer extends Product { // IS-A Product

    // HAS-A via composição:
    private Motherboard motherboard;
    private ComputerCase computerCase;
    private Monitor monitor;

    public PersonalComputer(String make, String model,
        Motherboard mb, ComputerCase cc, Monitor mon) {
        super(make, model);
        this.motherboard = mb;
        this.computerCase = cc;
        this.monitor = mon;
    }
    // Encapsula as partes — chamador não precisa conhecê-las
    public void powerUp() {
        computerCase.pressPowerButton();
        motherboard.loadProgram("Boot");
    }
}
```

■ Encapsulamento via Composição

O código chamador não precisa saber nada sobre Motherboard, ComputerCase ou Monitor para ligar o computador. Ele apenas chama `powerUp()` — as partes ficam ocultas.

Por que preferir Composição?

A herança tem um problema sutil: ela pode **quebrar o encapsulamento** da superclasse. Subclasses frequentemente precisam conhecer os detalhes internos do pai para funcionar corretamente — criando um acoplamento rígido que torna mudanças futuras muito custosas. Além disso, a herança é inflexível: adicionar ou remover uma classe da hierarquia impacta todas as subclasses.

Outro problema clássico: e se um novo requisito não se encaixar perfeitamente na hierarquia existente? Imagine que `Product` tem campos de dimensões físicas (largura, altura, profundidade). Faz sentido para um produto físico — mas e para um `DigitalProduct` (software, e-book)? Herdar dimensões físicas seria errado. Composição resolve isso elegantemente: só quem precisa das dimensões as inclui.

```
// Problema com herança pura:
public class Product { int width, height, depth; } // dimensões físicas
public class DigitalProduct extends Product { }    // ■ não tem dimensões!

// Solução com composição:
public class Dimensions { int width, height, depth; }
public class Product { String model, manufacturer; } // genérico

public class PhysicalProduct extends Product {
    private Dimensions dimensions; // HAS-A — só quem precisa usa
}
public class DigitalProduct extends Product {
    private String downloadUrl;    // sem dimensões — correto!
}
```

■ Flexibilidade da Composição

Design mais flexível: adicionar `DigitalProduct` não quebra nenhum código existente. Com herança pura, seria necessário refatorar `Product` inteiro.

3

Encapsulamento

Proteção de dados, acesso controlado e integridade do estado

Por que Encapsular?

Sem encapsulamento, qualquer parte do programa pode modificar o estado interno de um objeto diretamente — sem nenhuma validação ou controle. Isso cria dois problemas graves: primeiro, estados inválidos (ex: `health = -999` em um jogo); segundo, acoplamento excessivo — renomear um campo quebra todo o código que o referencia diretamente.

O encapsulamento age como uma **caixa-preta**: a implementação interna pode mudar completamente sem afetar quem usa a classe, desde que a interface pública (os métodos públicos) permaneça a mesma. Isso é o que permite que você melhore, refatore e otimize o código interno sem quebrar nada.

```
// ■ SEM encapsulamento – frágil e perigoso
public class Player {
    public String name;
    public int    health;    // qualquer um pode colocar -999
    public String weapon;
}
Player p = new Player();
p.health = -100; // ← inválido, mas nada impede!

// ■ COM encapsulamento
public class Player {
    private String name;
    private int    health; // só acessível via métodos controlados
    private String weapon;

    public Player(String name, int health, String weapon) {
        this.name     = name;
        this.health = Math.max(0, Math.min(100, health)); // valida!
        this.weapon = weapon;
    }

    public void loseHealth(int damage) {
        health = Math.max(0, health - damage); // nunca fica negativo
    }

    public int healthRemaining() { return health; }
}
```

Princípios do Encapsulamento

Seguir esses princípios garante que suas classes sejam seguras, previsíveis e fáceis de manter. São regras simples, mas que fazem uma diferença enorme na qualidade do código.

- | | |
|---------------------------------|--|
| private em tudo | Declare todos os campos como <code>private</code> — sem exceção. Nunca <code>public</code> . |
| Construtor inicializador | Crie construtores que inicializam o objeto com valores válidos, rejeitando dados inválidos desde o início. |
| Setters com parcimônia | Use setters apenas quando o campo realmente precisa mudar após a criação. Prefira imutabilidade. |

Exponha o mínimo	Torne public apenas o que o chamador precisa usar. Métodos auxiliares internos devem ser private.
Getters podem ser public	Retornar dados é seguro; modificar estado sem validação não é. Getters geralmente são public, setters nem sempre.
Interface pública = contrato	Os métodos públicos são o contrato com o mundo externo. Mudanças neles quebram código cliente.

■ Regra de Ouro

Encapsulamento = controle total sobre o estado do objeto. Ninguém de fora quebra as suas regras de negócio.

4**Polimorfismo**

Muitas formas — comportamento determinado em runtime

O que é Polimorfismo?

Polimorfismo vem do grego *polys* (muitos) + *morphe* (formas). Em Java, significa que o mesmo código pode se comportar de maneiras diferentes dependendo do tipo real do objeto em runtime. O tipo *declarado* da variável determina quais métodos você pode chamar (verificado em compile-time); o tipo *real* do objeto determina qual implementação é executada (decidido em runtime pela JVM).

Para o polimorfismo funcionar, é necessária uma relação de herança (IS-A): o tipo real deve ser igual ou uma subclasse do tipo declarado. A JVM usa uma tabela virtual de métodos (*vtable*) para encontrar a implementação correta em runtime — esse mecanismo é chamado de **dynamic dispatch** ou **late binding**.

O poder do polimorfismo está na **extensibilidade**: você pode adicionar um novo subtipo (ex: um novo gênero de filme: Horror, Romance, Thriller) sem modificar nenhuma linha de código existente que trabalhe com o tipo base. O código genérico continua funcionando para o novo tipo automaticamente.

```
public class Movie {
    private String title;
    public Movie(String title) { this.title = title; }
    public void watchMovie() {
        System.out.println(title + " [" + getClass().getSimpleName() + "]");
        System.out.println("... enredo generico ...");
    }
}

public class Adventure extends Movie {
    @Override public void watchMovie() {
        super.watchMovie();
        System.out.println("... cena de acao ... salto ...");
    }
}

public class Comedy extends Movie {
    @Override public void watchMovie() {
        super.watchMovie();
        System.out.println("... punchline ...");
    }
}

// POLIMORFISMO: tipo declarado = Movie, tipo real = Adventure
Movie movie = Movie.getMovie("Adventure", "Jaws");
movie.watchMovie(); // executa Adventure.watchMovie() — não Movie!
```

■ Dynamic Dispatch

O tipo declarado é `Movie` — mas a JVM decide em runtime qual implementação chamar com base no tipo real do objeto. O compilador só verifica se `watchMovie()` existe em `Movie`.

Código Genérico e Factory Method

O padrão **Factory Method** usa polimorfismo para criar objetos sem expor o tipo concreto ao chamador. Um método estático recebe um tipo como `String` (ou `enum`) e retorna o subtipo correto. O código cliente recebe um `Movie` — sem saber se é `Adventure`, `Comedy` ou `ScienceFiction`.

O resultado é um main de **duas linhas** que suporta N gêneros diferentes: adicionar um novo gênero exige apenas criar a subclasse e adicionar um case no switch do factory — o loop no main não muda.

```
// Factory method dentro de Movie:
public static Movie getMovie(String type, String title) {
    return switch (type.toUpperCase().charAt(0)) {
        case 'A' -> new Adventure(title);
        case 'C' -> new Comedy(title);
        case 'S' -> new ScienceFiction(title);
        default -> new Movie(title);
    };
}

// main genérico – nunca muda mesmo com novos gêneros:
String[] types = {"Adventure", "Comedy", "ScienceFiction"};
String[] titles = {"Indiana Jones", "The Mask", "Star Wars"};
for (int i = 0; i < types.length; i++) {
    Movie movie = Movie.getMovie(types[i], titles[i]); // compile: Movie
    movie.watchMovie(); // runtime: executa o subtipo correto
}
```

5

instanceof & Casting

Testar tipo em runtime, pattern matching (JDK 16+) e var (JDK 10+)

O operador instanceof

O operador instanceof testa se um objeto é uma instância de um determinado tipo em runtime. Ele é necessário quando você tem uma variável do tipo base (ex: Movie) e precisa acessar métodos específicos de um subtipo (ex: Adventure.fightScene()) que não existem no tipo base.

A forma clássica exige dois passos: testar com instanceof e depois fazer um cast explícito. O **Pattern Matching**, introduzido no JDK 16, combina os dois em um único passo: se instanceof retornar true, a variável já é automaticamente tipada como o subtipo — sem cast manual. O JDK 21 expandiu isso para switch, tornando o código ainda mais limpo.


```
// Forma clássica – cast explícito:
Movie unknownMovie = Movie.getMovie("Adventure", "Jaws");
if (unknownMovie instanceof Adventure) {
    Adventure adv = (Adventure) unknownMovie; // cast manual
    adv.fightScene();
}

// Pattern Matching (JDK 16+) – sem cast:
if (unknownMovie instanceof Adventure adv) {
    adv.fightScene(); // 'adv' já é Adventure – limpo e seguro!
}

// Switch com pattern matching (JDK 21):
switch (unknownMovie) {
    case Adventure a      -> a.fightScene();
    case Comedy c         -> c.tellJoke();
    case ScienceFiction s -> s.warpSpeed();
    default                -> unknownMovie.watchMovie();
}
```

■ Pattern Matching

Prefira pattern matching ao cast manual — mais conciso e elimina o risco de `ClassCastException` acidental. Se `instanceof` retorna `true`, o cast é seguro por definição.

var e LVTI — Local Variable Type Inference

Introduzido no Java 10, `var` é uma palavra-chave especial que instrui o compilador a **inferir o tipo** da variável local a partir da expressão de inicialização. Isso reduz verbosidade — especialmente com tipos genéricos longos como `ArrayList<Map<String, List<Integer>>>`. O tipo ainda é resolvido em compile-time — Java não vira uma linguagem dinâmica.

Restrições importantes: `var` só funciona em **variáveis locais** com atribuição na mesma linha. Não pode ser usado em campos de classe, parâmetros de métodos, tipos de retorno, ou com `null` literal (o compilador não consegue inferir o tipo de `null` sozinho).

```
// Com tipo explícito – verboso:
ArrayList<String> list = new ArrayList<String>();

// Com var – mais limpo, mesmo resultado:
var list = new ArrayList<String>(); // compilador infere ArrayList<String>

// var em for-each:
for (var entry : map.entrySet()) {
    System.out.println(entry.getKey() + "=" + entry.getValue());
}

// ■ Usos INVÁLIDOS de var:
// var field = 10;           // campo de classe – ERRO
// public var getName() {}  // tipo de retorno – ERRO
// void method(var x) {}    // parâmetro – ERRO
// var x;                   // sem atribuição – ERRO
// var x = null;            // não infere tipo de null – ERRO

// Compile-time vs Runtime type com var:
var movie = Movie.getMovie("Comedy", "The Mask");
// compile: Movie (tipo inferido) | runtime: Comedy
```

■ var não é JavaScript!

var não torna Java dinâmico — o tipo ainda é resolvido em compile-time pelo compilador. É apenas açúcar sintático para reduzir boilerplate sem perder segurança de tipo.

6

Packages & Import

Organizar classes, evitar conflitos e controlar visibilidade

O que é um Package?

Um **package** é um namespace que organiza classes relacionadas em grupos lógicos — semelhante a pastas em um sistema de arquivos (e de fato corresponde a diretórios). Packages evitam conflitos de nome: você pode ter duas classes chamadas `Date` em packages diferentes (`java.util.Date` e `java.sql.Date`) sem ambiguidade.

A convenção de nomes usa o domínio reverso da empresa como prefixo: `com.minhaempresa.projeto.modulo`. Isso garante unicidade global. Classes no mesmo package enxergam umas às outras com visibilidade **package-private** (sem modificador de acesso). O **FQCN** (Fully Qualified Class Name) é o nome completo com package: `java.util.Scanner`.

```
// package statement – PRIMEIRA linha do arquivo:
package com.minhaempresa.burger;

// import statements – vêm DEPOIS do package:
import java.util.ArrayList; // importa classe específica
import java.util.List;
import java.util.*;         // wildcard – importa todo o pacote (menos recomendado)

// Usando FQCN sem import – verboso mas sem ambiguidade:
java.util.Scanner sc = new java.util.Scanner(System.in);

// Regras importantes:
// 1. Só UM package statement por arquivo
// 2. package ANTES de qualquer import
// 3. Sem package = default package (evite em projetos reais!)
// 4. Classes em packages diferentes precisam de import ou FQCN
// 5. java.lang é importado automaticamente (String, Object, Math...)
```

■ FQCN e Import

FQCN = package + nome da classe. Ex: java.util.Scanner. Use import para evitar escrever o FQCN toda vez. Prefira imports específicos a wildcards — mais legível.

7

Mini-Projects

Smart Kitchen (Composição), Printer (Encapsulamento), Car (Polimorfismo)

Os mini-projects a seguir aplicam os três pilares em cenários independentes antes do desafio master. Cada um foca em um pilar principal, mas os outros dois aparecem de forma natural.

Mini-Project A

Smart Kitchen — Composição

Crie a classe SmartKitchen com três eletrodomésticos como campos de composição: CoffeeMaker, Refrigerator e DishWasher. Cada um tem um campo hasWorkToDo (boolean) e seu próprio método de trabalho. SmartKitchen expõe métodos como addWater(), pourMilk() e loadDishwasher() que delegam para os componentes internos sem expô-los diretamente.

```
public class SmartKitchen {
    private CoffeeMaker coffeeMaker = new CoffeeMaker();
    private Refrigerator refrigerator = new Refrigerator();
    private DishWasher dishWasher = new DishWasher();

    public void addWater() { coffeeMaker.setHasWorkToDo(true); }
    public void pourMilk() { refrigerator.setHasWorkToDo(true); }
    public void loadDishwasher() { dishWasher.setHasWorkToDo(true); }

    // Delegação – sem acesso direto às partes:
    public void doKitchenWork() {
        coffeeMaker.doWork();
        refrigerator.doWork();
        dishWasher.doWork();
    }
}
```

Mini-Project B**Printer — Encapsulamento na Prática**

Crie uma classe `Printer` que controla nível de toner (0–100%) e contagem de páginas. O construtor recebe `tonerAmount` e `duplex`. O método `addToner(amount)` retorna -1 se o novo nível sair do range 0–100. O método `printPages(pages)` calcula folhas físicas (`duplex =  $\lceil \text{pages} / 2 \rceil$`) e acumula em `pagesPrinted`. Todos os campos devem ser `private`.

```
public class Printer {
    private int tonerLevel;
    private int pagesPrinted;
    private boolean duplex;

    public Printer(int tonerAmount, boolean duplex) {
        this.tonerLevel = Math.max(0, Math.min(100, tonerAmount));
        this.duplex = duplex;
        this.pagesPrinted = 0;
    }

    public int addToner(int amount) {
        int newLevel = tonerLevel + amount;
        if (newLevel < 0 || newLevel > 100) return -1;
        tonerLevel = newLevel;
        return tonerLevel;
    }

    public int printPages(int pages) {
        int sheets = duplex ? (int) Math.ceil(pages / 2.0) : pages;
        if (duplex) System.out.println("Duplex printing.");
        pagesPrinted += sheets;
        return sheets;
    }

    public int getPagesPrinted() { return pagesPrinted; }
}
```

Mini-Project C

Car Polymorphism Challenge

Crie a hierarquia de carros: classe base Car com startEngine(), drive() e runEngine() (protected). O método drive() chama runEngine() — polimorfismo automático. Implemente GasPoweredCar, ElectricCar e HybridCar, cada uma sobrescrevendo startEngine() e runEngine() com comportamento único. No main, instancie um de cada e execute drive() — o polimorfismo cuida do resto.

```
public class Car {
    private String description;
    public Car(String desc) { this.description = desc; }

    public void startEngine() {
        System.out.println(getClass().getSimpleName() + ": engine starts.");
    }
    public final void drive() { runEngine(); } // template — polimorfismo!
    protected void runEngine() { System.out.println("Generic engine."); }
}
public class ElectricCar extends Car {
    private int batteryLevel;
    public ElectricCar(String desc, int battery) {
        super(desc); this.batteryLevel = battery;
    }
    @Override public void startEngine() {
        System.out.println("ElectricCar: silent power-on. Battery: "
            + batteryLevel + "%");
    }
    @Override protected void runEngine() {
        System.out.println("ElectricCar: electric motor running silently.");
    }
}
// GasPoweredCar e HybridCar seguem o mesmo padrão
```

8

Mini-Project Master — Bill's Burger

OOP completo: Composição + Encapsulamento + Polimorfismo

Bill's Burger é o projeto integrador desta apostila — ele aplica os três pilares da OOP de forma real e progressiva. É o tipo de exercício que aparece em entrevistas técnicas porque exige pensar em hierarquia, delegação e polimorfismo ao mesmo tempo.

Arquitetura da Solução

A hierarquia começa com Item — a superclasse genérica que representa qualquer item do cardápio (nome, tipo, preço, tamanho). Burger herda de Item e adiciona um array de toppings (coberturas

extras). DeluxeBurger herda de Burger e tem preço fixo — os toppings são gratuitos.

MealOrder usa **composição**: tem um Burger, um Item (bebida) e um Item (acompanhamento). O método getTotalPrice() chama getAdjustedPrice() em cada componente — **polimorfismo puro**: o mesmo método se comporta diferente para Burger (soma toppings), DeluxeBurger (retorna fixo) e Item (ajusta por tamanho).

Item	Object	name, type, price, size getAdjustedPrice() por tamanho
Burger	Item	toppings[3], toppingCount getAdjustedPrice() soma extras
DeluxeBurger	Burger	toppings[5], preço fixo getAdjustedPrice() = fixo
MealOrder	Object (composição)	burger, drink, sideltem getTotalPrice(), printItemizedList()

Implementação — Item e Burger

```
public class Item {
    private String name, type;
    private double price;
    private String size = "MEDIUM";

    public Item(String name, String type, double price) {
        this.name = name; this.type = type; this.price = price;
    }
    public double getBasePrice() { return price; }
    public double getAdjustedPrice() {
        return switch (size) {
            case "SMALL" -> price * 0.75;
            case "LARGE" -> price * 1.50;
            default      -> price;          // MEDIUM = preço base
        };
    }
    public void printItem() {
        System.out.printf("%-20s %8.2f%n", name, getAdjustedPrice());
    }
}

public class Burger extends Item {
    private Item[] toppings = new Item[3];
    private int    toppingCount = 0;

    public Burger(String type, double price) {
        super(type + " Burger", "Burger", price);
    }
    public void addTopping(String name, double extraCost) {
        if (toppingCount < toppings.length)
            toppings[toppingCount++] = new Item(name, "Topping", extraCost);
    }
    @Override
    public double getAdjustedPrice() {
        double total = getBasePrice();
        for (var t : toppings) if (t != null) total += t.getBasePrice();
        return total; // preço base + soma dos toppings
    }
}
```

Implementação — DeluxeBurger e MealOrder

```
// DeluxeBurger: preço fixo, toppings gratuitos
public class DeluxeBurger extends Burger {
    public DeluxeBurger() { super("Deluxe", 8.50); }

    @Override
    public void addTopping(String name, double cost) {
        super.addTopping(name, 0); // custo zero – preço é fixo!
    }
    @Override
    public double getAdjustedPrice() { return getBasePrice(); } // FIXO
}

// MealOrder: composição + polimorfismo
public class MealOrder {
    private Burger burger;
    private Item drink;
    private Item sideItem;

    // Construtor padrão com valores default
    public MealOrder() {
        this(new Burger("Regular", 4.00),
             new Item("Coke", "Drink", 1.50),
             new Item("Fries", "Side", 1.00));
    }
    public MealOrder(Burger b, Item d, Item s) {
        this.burger = b; this.drink = d; this.sideItem = s;
    }
    public void addToppingToBurger(String name, double cost) {
        burger.addTopping(name, cost); // delega – encapsulamento!
    }
    public void printItemizedList() {
        burger.printItem(); drink.printItem(); sideItem.printItem();
        System.out.printf("%-20s %8.2f%n", "TOTAL", getTotalPrice());
    }
    public double getTotalPrice() {
        // POLIMORFISMO: cada tipo executa sua versão de getAdjustedPrice()
        return burger.getAdjustedPrice() // Burger soma toppings
            + drink.getAdjustedPrice() // Item ajusta por tamanho
            + sideItem.getAdjustedPrice();
    }
}
```

■ Polimorfismo em ação

getAdjustedPrice() em Burger soma toppings; em DeluxeBurger retorna preço fixo; em Item regular ajusta pelo tamanho. Mesmo método, três comportamentos — polimorfismo puro.

Resumo — O que você aprendeu nesta apostila

Composição (HAS-A)	Objetos formados por outros objetos. Mais flexível que herança. Prefira composição na dúvida.
Encapsulamento	Campos private, construtores com validação, setters com parcimônia. Caixa-preta: muda por dentro sem quebrar por fora.
Polimorfismo	Override + tipo real em runtime (dynamic dispatch). Código genérico extensível sem modificações.
instanceof	Testa tipo real em runtime. Pattern matching (JDK 16+) elimina cast manual. Switch pattern (JDK 21).
var / LVTI	Inferência de tipo local (JDK 10+). Só em variáveis locais com atribuição. Não é dinâmico — compile-time.
Packages	Namespace para organizar classes. Domínio reverso por convenção. import ou FQCN para usar entre packages.
Factory Method	Método estático que cria e retorna o subtipo correto. Chamador recebe o tipo base — não sabe o tipo real.
Bill's Burger	Item → Burger → DeluxeBurger (herança) + MealOrder com composição. getAdjustedPrice() = polimorfismo.